API and Data Reference

Status date: April 29, 2026

This document is the contract between input files, the in-memory scheduling state, and the export artifacts. It is also the reference for the public Dart surface a future feature would extend.

1. API Status

The Omnilore Scheduler is a **client-side application**. There is no HTTP API, no database, no JSON schema, no server-side state. "API" in this document means:

- The format of the **input text files** the user uploads (course.txt, people.txt).
- The format of the **bundled save file** the app produces.
- The grammar of the **scheduler-state section** inside that bundle.
- The signatures of the **public Dart methods** on Scheduling that a new pipeline stage or export would extend.
- The keys the browser reads/writes in **localStorage** for autosave.
- The filenames and shapes of the **four export menu actions**.

2. Input Files

2.1 Course file

Loader: lib/store/courses.dart (Courses.loadCourses / loadCoursesFromBytes).
Format: UTF-8 plain text, **tab-delimited**, three columns per row, one row per course.

<course_code>\t<course_name>\t<reading_material>\n

Example (excerpt from test/resources/course.txt):

ASA Easternization: Asia's Rise and America's Decline Easternization: Asia's Rise and America's Decline . . . , by Gideon Rachman
BAD Bad Girls Throughout History: 100 Remarkable Women . . . Bad Girls Throughout History: 100 Remarkable Women . . . , by Ann Shen
BIG Big Ones The Big Ones: How Natural Disasters Have Shaped Us, by Dr. Lucy Jones

Field rules:

Column	Meaning	Notes
1	code	Three-letter (or short) unique course identifier. Whitespace-trimmed.
2	name	Full course name. Whitespace-trimmed.

3	reading	Recommended reading. May be empty (No Common Reading).

Errors thrown by Courses.loadCourses:

- MalformedInputException — wrong number of tab-separated tokens.
- DuplicateCourseCodeException — same code appears twice.
- File-system errors propagate as the platform's default exception (desktop) or as caller-handled byte errors (web).

2.2 People file

Loader: lib/store/people.dart (People.loadPeople / loadPeopleFromBytes). **Format:** UTF-8 plain text, **tab-delimited**, exactly **21 columns** per row, one row per person.

<lastName>\t<firstName>\t<phone>\t<numClassWanted>\t<avail0>…<avail9>\t<choice1>…<choice6>\t<submissionOrder>\n

Column-by-column:

Cols	Meaning	Allowed values
1	Last name	non-empty string
2	First name	non-empty string
3	Phone	string (display format like 545-2286)
4	Number of classes wanted	integer 0–6, or empty (treated as "no submission")
5–14	Availability for days 1–5 (Mon–Fri)	'', '1', '2', or '3' (see §2.3)
15–20	First choices then backups, in priority order	course codes that exist in the course file; up to 6 codes total
21	Submission order	integer; smaller = earlier submission = higher placement priority

If column 4 fails to parse as an integer, the row is treated as a non-submitter (defaults: zero classes wanted, fully available, no choices). Otherwise:

- numClassWanted outside [0, 6] → InvalidNumClassWantedException.
- An availability value outside '', '1', '2', '3' → UnrecognizedAvailabilityException.
- A duplicate course code on the same person → DuplicateClassSelectionException.
- Choices listed with numClassWanted == 0 → ListingWhenWantingZeroException.

- Wanting more classes than listed → WantingMoreThanListedException.
- Choosing a course code that does not exist in course.txt → InconsistentCourseAndPeopleException.
- A duplicate person record (same firstName + lastName) → DuplicateRecordsException.
- Wrong column count → MalformedInputException.

The full set of exception types lives in lib/model/exceptions.dart.

2.3 Availability encoding

Each of the ten Mon–Fri columns covers two **terms** (1 & 3 and 2 & 4), each split into AM and PM. The encoded value tells the scheduler which terms the person is **unavailable**:

Value	Meaning
'' (empty)	Available in both Term 1 & 3 *and* Term 2 & 4 for that day (AM and PM).
'1'	Unavailable in Term 1 & 3 (AM and PM that day); available in Term 2 & 4.
'2'	Unavailable in Term 2 & 4 (AM and PM that day); available in Term 1 & 3.
'3'	Unavailable in both terms that day.

The parser flips matching slots in the 20-element availability array (true = available). The slot indexing matches §5 below.

3. Bundled Save File

Builder/parser: lib/io/bundled_state.dart (buildBundledStateContent, parseBundledStateContent).

A **Save** or **Save As** writes a single text file in this layout:

```
CourseFile:
<verbatim contents of the loaded course.txt, including its trailing newline>
PeopleFile:
<verbatim contents of the loaded people.txt, including its trailing newline>
Setting:
<scheduler-state grammar — see §4>
```

Parser invariants:

- The file *must* start with CourseFile:\n. If not, the parser treats the entire file as a legacy "state-only" file and the caller is expected to have course and people data already loaded.
- The string \nPeopleFile:\n *must* appear after CourseFile:.
- The string \nSetting:\n *must* appear after PeopleFile:. (Setting: is the first line of the state grammar.)

A FormatException is thrown if those markers are missing.

The bundled save preserves the exact bytes of the original course and people files, so a stakeholder can hand a single file to an admin and reproduce the entire session.

4. Scheduler-State Grammar (Setting: section)

Generated by Scheduling.exportStateToString() and consumed by Scheduling.loadState().

```
Setting:
Min: <int>
Max: <int>

Course size:
<courseCode>: <minSize>,<maxSize>
…

Drop:
<courseCode>
…

Limit:
<courseCode>
…

Split:
Course: <courseCode>
Cluster: <person1>,<person2>,…
Cluster: <person>,…

Schedule:
<courseCode>: <timeIndex>
…

Coordinator:
<courseCode>: equal,<personA>,<personB>
<courseCode>: unequal,<mainCoordinator>,<coCoordinator>
…
```

Section meanings:

Section	Field on Scheduling	Notes
Setting:	courseControl.getGlobal{Min,Max}ClassSize()	Global default class-size bounds.
Course size:	courseControl.getCustomSizeClasses() + getMinClassSize / getMaxClassSize	Per-course overrides; courses without an override use the global bounds.

Drop:	courseControl.getDropped()	Courses the admin chose to drop instead of run.
Limit:	getSplitMode(course) == SplitMode.limit	Courses the admin chose to **cap at max size** rather than split.
Split:	splitControl.getHistory()	Implemented (committed) splits, in chronological order. Each Cluster: row lists the people assigned to one resulting class.
Schedule:	scheduleControl.scheduledTimeFor(course)	Time-slot index 0–19 (see §5). Courses without an entry are unscheduled.
Coordinator:	courseControl.getCoordinators(course)	equal,<A>,<B> for two equal co-coordinators; unequal,<C>,<CC> for one main coordinator plus a co-coordinator.

Backwards compatibility: a state-only file (without the CourseFile: / PeopleFile: prefixes) loads against existing in-memory course and people data, matching the format the app produced before D4.

5. Time-Slot Index Reference

Scheduling.getTimeslotDescription(int) maps the 20 time-slot indices used everywhere in the schedule grammar:

Index	Term	Day	AM/PM
0	1 & 3	Mon	AM
1	1 & 3	Mon	PM
2	1 & 3	Tue	AM
3	1 & 3	Tue	PM
4	1 & 3	Wed	AM
5	1 & 3	Wed	PM
6	1 & 3	Thu	AM
7	1 & 3	Thu	PM
8	1 & 3	Fri	AM
9	1 & 3	Fri	PM
10	2 & 4	Mon	AM
11	2 & 4	Mon	PM
12	2 & 4	Tue	AM

13	2 & 4	Tue	PM
14	2 & 4	Wed	AM
15	2 & 4	Wed	PM
16	2 & 4	Thu	AM
17	2 & 4	Thu	PM
18	2 & 4	Fri	AM
19	2 & 4	Fri	PM

Indices below 10 are Term 1 & 3; ≥10 are Term 2 & 4. Even indices are AM; odd are PM. Day = (index % 10) / 2 (Mon=0, Tue=1, ..., Fri=4).

getTimeslotDescription throws InvalidArgument for indices outside [0, 19].

6. Public Scheduling Surface (extension points)

These are the most useful methods on Scheduling for adding a new feature, an export, or a pipeline stage. All are defined in lib/scheduling.dart.

Method	Signature	Purpose
getStateOfProcessing()	StateOfProcessing	Current pipeline stage. UI-gating is wired off this.
compute(Change)	void	Recomputes affected modules after a UI change. Pass Change(course: true, people: false, drop: false, schedule: false, misc: false) etc.
loadCoursesFromBytes / loadPeopleFromBytes	Future<int>	Web-safe loaders that bypass dart:io.
outputRosterCC / outputRosterCCToString	path → file / String	Final roster with (C) and (CC) labels. Requires output state.
outputRosterPhone / outputRosterPhoneToString	path → file / String	Roster with phone numbers. Available in coordinator or output state.
outputMM / outputMMToString	path → file / String	Tab-delimited mail-merge, one row per person. Requires output state.
outputUnmetWants / outputUnmetWantsToString	path → file / String	Total unmet count + per-person gaps. Requires output state.
exportStateToString()	String	Produces the Setting:

		grammar. Used inside the bundled save.
exportState(path) / loadState(path)	path → file / file → state	Legacy state-only save and loader. Bundled save is the preferred path.
getTimeslotDescription(int)	String	Human-readable slot label; see §5.

When adding a new export:

1. Add outputXyzToString() returning the rendered text.
2. Gate it on getStateOfProcessing() so it can't run before the pipeline reaches a sensible state.
3. Add a UI handler in lib/widgets/screen.dart that routes through exportTextFile() (lib/io/export_text_file.dart) so it picks up showSaveFilePicker on web and the file-picker on desktop.
4. Add a regression test under test/.

7. Output Artifacts

Default suggested filenames are built by lib/io/default_filename.dart and passed from lib/widgets/screen.dart. They include a timestamp suffix (<base>_YYYY-MM-DD_HHMM.txt) so repeated exports do not overwrite each other. On showSaveFilePicker-supporting browsers the user can rename; otherwise the file lands in the browser's downloads folder under the suggested name.

Export	Suggested filename	Method	Required state	Shape
Early roster	early_roster_YYYY-MM-DD_HHMM.txt	outputRosterPhoneToString()	coordinator or output	<course>\t<timeslot> headers, then assigned names with phone numbers.
Final roster	final_roster_YYYY-MM-DD_HHMM.txt	outputRosterCCToString()	output	Roster with (C) and (CC) labels; canonical post-handoff artifact.
Mail merge	mail_merge_YYYY-MM-DD_HHMM.txt	outputMMToString()	output	Tab-delimited; one row per person; columns include person identity plus up

				to nine class slots (supports 6+ wanted).
Unmet wants	unmet_wants_YYYY-MM-DD_HHMM.txt	outputUnmetWantsToString()	any state with summaries available	Per-person Wants: N, Unmet: N, Assigned: … blocks; intended for triage of admin gaps.

Mail-merge column note (from CHANGES.md). The web/D4+ mail-merge has nine extra columns per line versus the legacy desktop format, to support members wanting more than three classes.

8. Browser Storage Keys

Defined in lib/io/autosave_store_web.dart. All values are plain text; sizes are bounded by the bundled-state file size (typically <1 MB for the canonical 24/267 fixture).

Key	Set by	Read by
omnilore_autosave	autosave debounced after a state change	restore prompt on app load
omnilore_autosave_time	same as above (ISO-8601 timestamp)	restore prompt timestamp display
omnilore_hardsave	explicit Save (when Save As falls back to localStorage)	restore prompt
omnilore_hardsave_time	as above	restore prompt timestamp
omnilore_course_data	autosave path	seed for the course store on restore
omnilore_people_data	autosave path	seed for the people store on restore

The autosave store has a debounce timer to keep localStorage writes off the hot path of every keystroke / click.

9. Errors and User Messages

The full exception list is in lib/model/exceptions.dart. The most user-facing ones, with the message a stakeholder will see:

Exception	Triggered by	User-friendly meaning
MalformedInputException	Wrong column count on a	"Line N: expected X

	course or people row	columns but got Y."
DuplicateCourseCodeException	Same course code twice in course.txt	"The course file contains duplicate course codes: ."
InvalidNumClassWantedException	Person column 4 outside 0–6	"People file specifies invalid number of classes wanted: at line ."
UnrecognizedAvailabilityException	Availability cell ≠ '', '1', '2', '3'	"People file specifies unrecognized availability value: at line ."
DuplicateClassSelectionException	Same course code twice on one person	"A class is chosen more than once: at line ."
WantingMoreThanListedException	Wants > listed	"Member wants more classes than listed at line ."
ListingWhenWantingZeroException	numClassWanted = 0 but choices present	"Listing classes despite wanting zero at line ."
InconsistentCourseAndPeopleException	A person references an unknown course code	The exception message names the offending code.
DuplicateRecordsException	Two rows with the same first/last name	" has more than one record."
InvalidArgument	Time-slot index outside 0–19	"Invalid time index."

When the parser raises any of the file-level exceptions, the in-memory store is **cleared** (the file load is atomic — partial loads are not allowed to leak through).

10. Test Fixtures (seed/test data notes)

File	Purpose
test/resources/course.txt	Canonical 24-course offering (production-shaped).
test/resources/people.txt	Canonical 267-person roster (production-shaped).
test/resources/course_split.txt, people_split.txt	Forces the split path.
test/resources/course_whitespace.txt	Whitespace-tolerance edge cases.
test/resources/people_schedule.txt	Drives schedule-stage tests.
test/resources/people_drop.txt	Drives drop-stage tests.
test/resources/malformed_*	Per-exception failure-path inputs.

Use these as your reference whenever a stakeholder reports a "the file won't load" issue — diff their file against the matching fixture and the column shape will jump out.